BT5110: Tutorial 2 — Entity - Relationship Model

Pratik Karmakar

School of Computing,
National University of Singapore

AY26/27 S1

The Problem

The Varsity International Network of Oenology wishes to computerise the management of the information about its members as well as to record the information they gather about various wines. Your company, Apasaja Private Limited, is commissioned by the Varsity International Network of Oenology to design and implement the relational schema of the database application. The organisation is big enough so that there could be several members with the same name. A card with a unique number is issued to identify each drinker. The contact address of each member is also recorded for the mailing of announcements and calls for meetings.

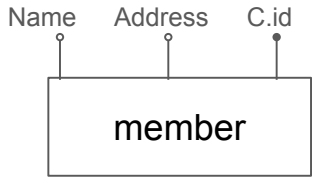
At most once a week, VINO organises a tasting session. At each session, the attending members taste several bottles. Each member records for each bottle his or her evaluation of the quality (very good, good, average, mediocre, bad, very bad) of each wine that she or he tastes. The evaluation may differ for the same wine from one drinker to another. Actual quality and therefore evaluation also varies from one to another bottle of a given wine. Every bottle that is opened during the tasting session is finished during that session.

Each wine is identified by its name ("Parade D'Amour"), appellation ("Bordeaux") and vintage (1990). Other information of interest about the wine is the degree of alcohol (11.5), where and by whom it has been bottled ("Mis en Bouteille par Amblard-Larolphie Negociant-Eleveur a Saint Andre de Cubzac (Gironde) - France"), the certification of its appellation if available ("Appellation Bordeaux Controlée"), and the country it comes from (produce of "France").

Generally, there are or have been several bottles of the same wine in the cellar. For each wine, the bottles in the wine cellar of VINO are numbered. For instance, the cellar has 20 bottles numbered 1 to 20 of a Semillon from 1996 named Rumbalara. For documentation purposes, VINO may also want to record wines for which it does not own bottles. The bottles are either available in the cellar or they have been tasted and emptied.

We first want to design an entity-relationship schema that most correctly and most completely captures the constraints expressed in the above description of the VINO application.

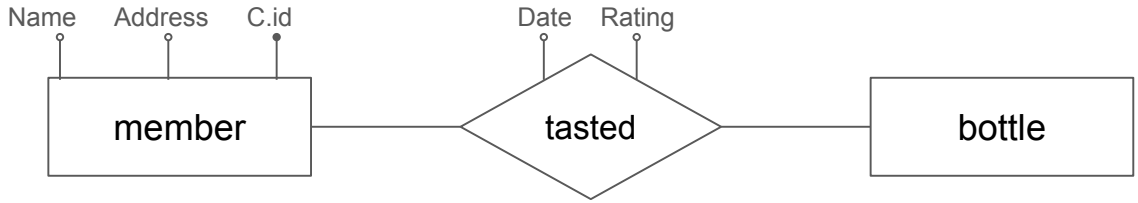
"The organisation is big enough so that there could be several **members** with the same **name**. **A card with a unique number is issued to identify each drinker.** The **contact address** of each member is also recorded for the mailing of announcements and calls for meetings."



member is an entity set with attributes: **Name**, **Address** and **Card Number (C.id)**, where C.id acts as the primary key.

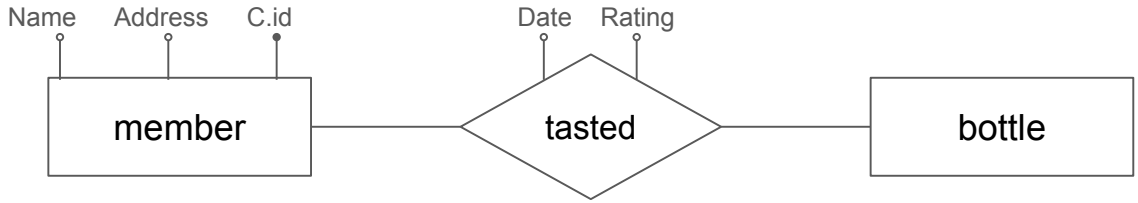
"At most once a week, VINO organises a tasting session. At each session, the **attending members** taste **several bottles**. Each member records for each bottle his or her **evaluation** of the quality (very good, good, average, mediocre, bad, very bad) of each wine that she or he tastes. The evaluation may differ for the same wine from one drinker to another. Actual quality and therefore evaluation also varies from one to another bottle of a given wine. Every bottle that is opened during the tasting session is finished during that session."

bottle is another entity set, and is related to **member** by a relationship called **tasted**: **Members taste bottles** in different sessions to give **ratings**.



"At most once a week, VINO organises a tasting session. At each session, the **attending members** taste **several bottles**. Each member records for each bottle his or her **evaluation** of the quality (very good, good, average, mediocre, bad, very bad) of each wine that she or he tastes. The evaluation may differ for the same wine from one drinker to another. Actual quality and therefore evaluation also varies from one to another bottle of a given wine. Every bottle that is opened during the tasting session is finished during that session."

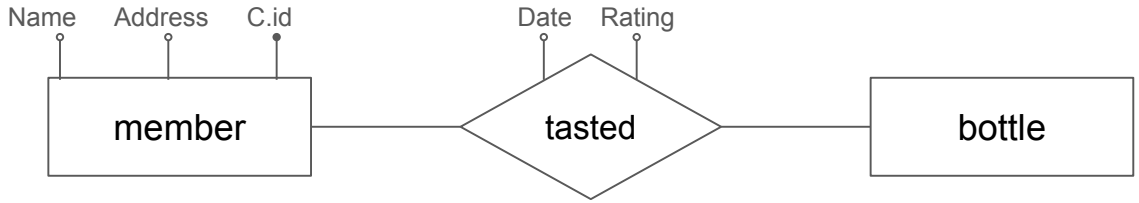
bottle is another entity set, and is related to **member** by a relationship called **tasted**: **Members taste bottles** in different sessions to give **ratings**.

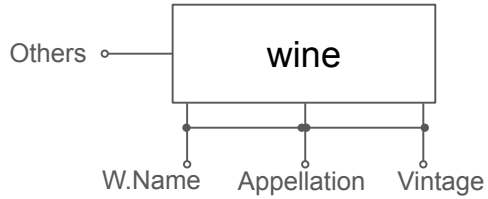


We do not yet know the attributes of the **bottle** entity. But essentially necessary attributes from **member** and **bottle** will be borrowed by the relationship **tasted.**

"Each wine is **identified by its name ("Parade D'Amour"), appellation ("Bordeaux") and vintage (1990)**. Other information of interest about the wine is the **degree of alcohol** (11.5), where and **by whom it has been bottled** ("Mis en Bouteille par Amblard-Larolphie Negociant-Eleveur a Saint Andre de Cubzac (Gironde) - France"), the **certification of its appellation** if available ("Appellation Bordeaux Controlée"), and the **country it comes from** (produce of "France")."

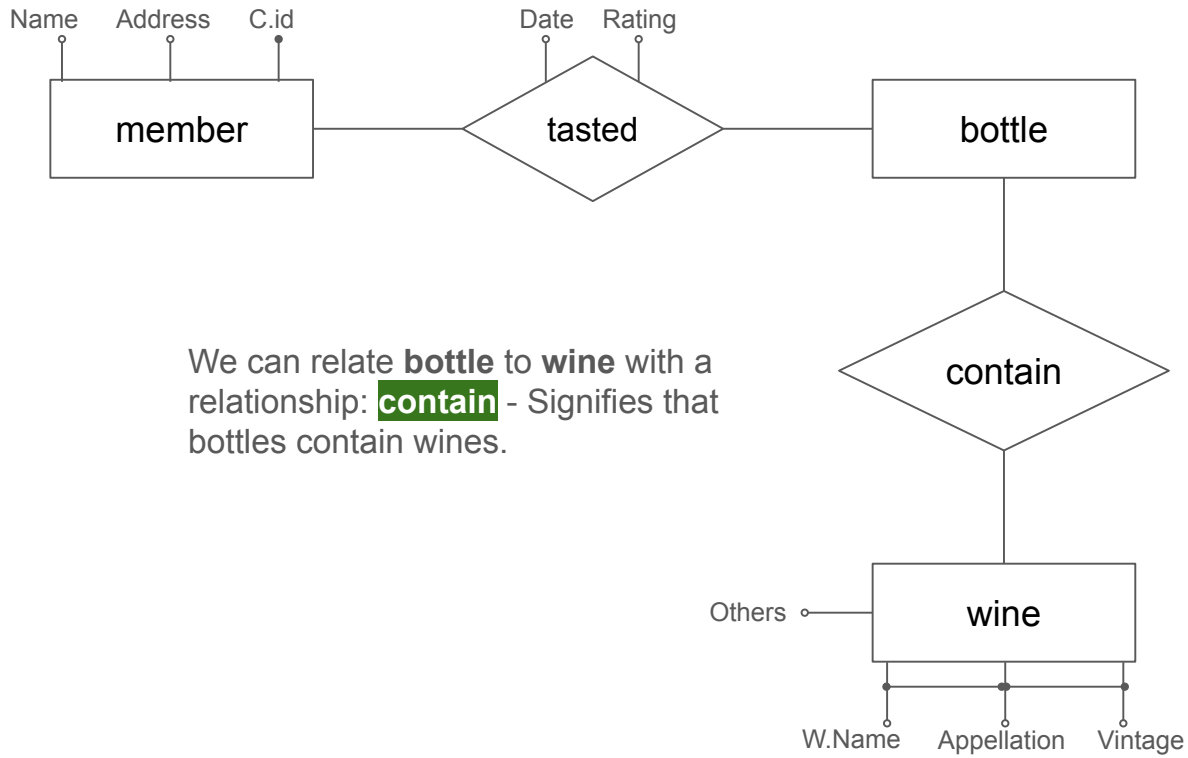
wine is another entity set with a **composite primary key** which consists of **Name, Appellation & Vintage.** It has other attributes: Alcohol_degree, Bottled_by, Appellation_cert and Country (denoted by **Others**).





"Each **wine** is **identified by its name ("Parade D'Amour"), appellation ("Bordeaux") and vintage (1990)**. Other information of interest about the wine is the **degree of alcohol** (11.5), where and **by whom it has been bottled** ("Mis en Bouteille par Amblard-Larolphie Negociant-Eleveur a Saint Andre de Cubzac (Gironde) - France"), the **certification of its appellation** if available ("Appellation Bordeaux Controlée"), and the **country it comes from** (produce of "France")."

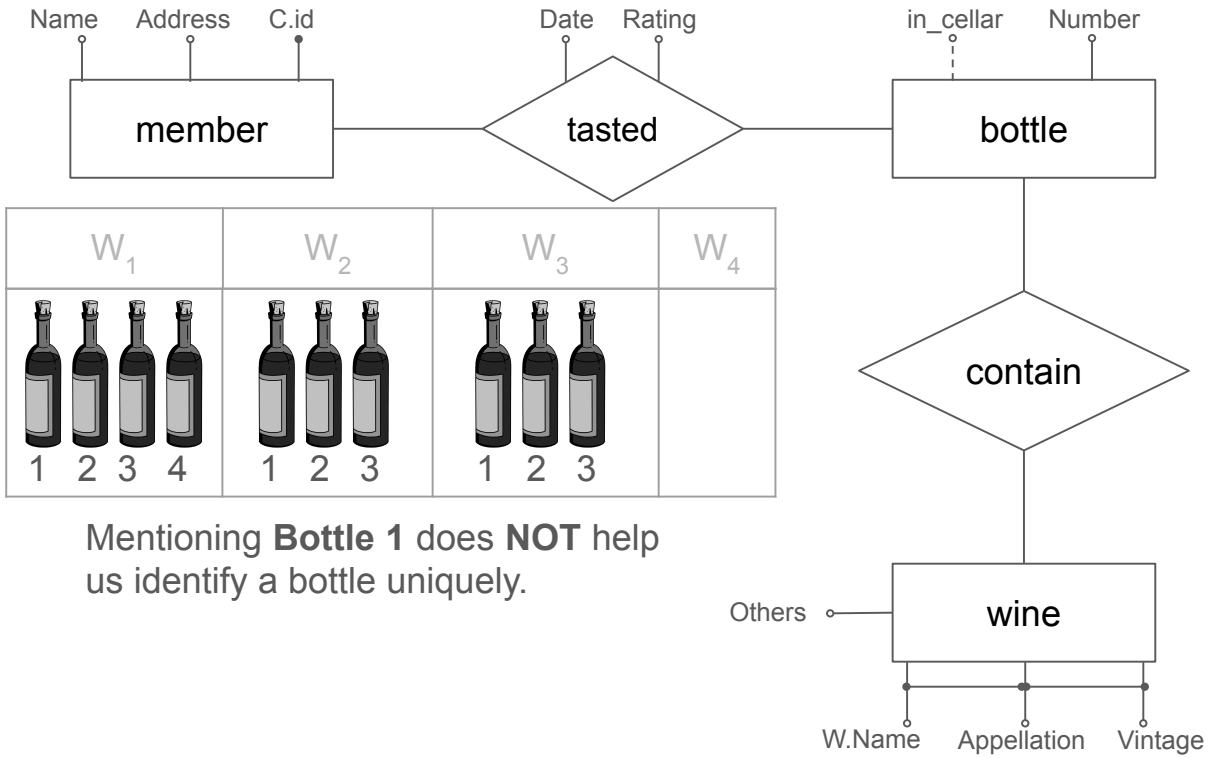
wine is another entity set with a **composite primary key** which consists of **Name, Appellation & Vintage.** It has other attributes: Alcohol_degree, Bottled_by, Appellation_cert and Country (denoted by **Others**).



We can relate **bottle** to **wine** with a relationship: **contain** - Signifies that bottles contain wines.

"Generally, there are or have been **several bottles of the same wine** in the cellar. **For each wine, the bottles in the wine cellar of VINO are numbered.** For instance, the cellar has 20 bottles numbered 1 to 20 of a Semillon from 1996 named Rumbalara. **For documentation purposes, VINO may also want to record wines for which it does not own bottles.** The bottles are **either available in the cellar or they have been tasted and emptied**."

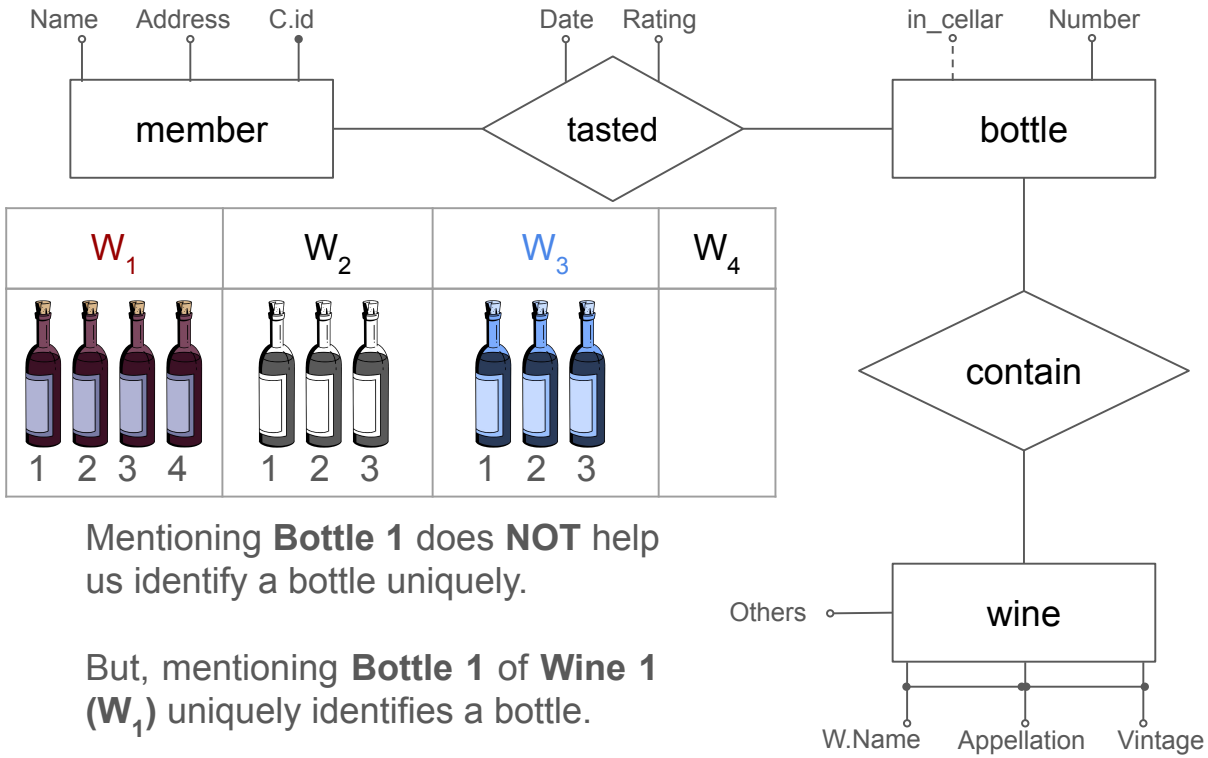
Bottles have **Number** and **Status** as its own attributes. But these are not enough to identify the bottles **UNIQUELY**.



Mentioning **Bottle 1** does **NOT** help us identify a bottle uniquely.

"Generally, there are or have been **several bottles of the same wine** in the cellar. **For each wine, the bottles in the wine cellar of VINO are numbered.** For instance, the cellar has 20 bottles numbered 1 to 20 of a Semillon from 1996 named Rumbalara. **For documentation purposes, VINO may also want to record wines for which it does not own bottles.** The bottles are **either available in the cellar or they have been tasted and emptied**."

Bottles have **Number** and **Status** as its own attributes. But these are not enough to identify the bottles **UNIQUELY**.

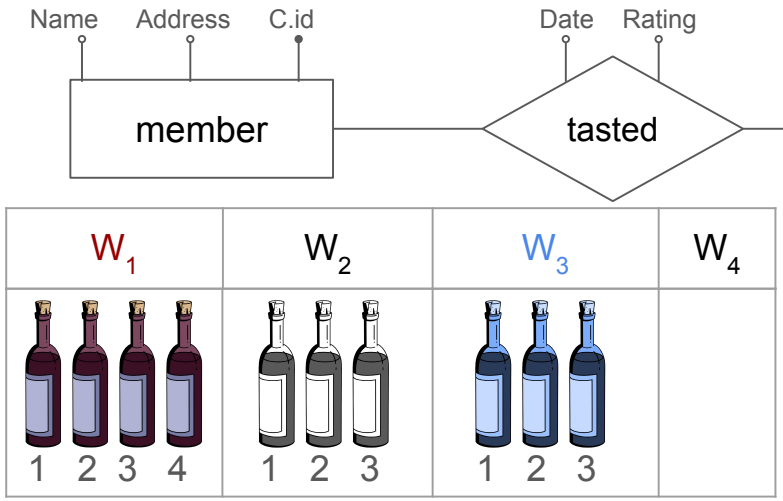


Mentioning **Bottle 1** does **NOT** help us identify a bottle uniquely.

But, mentioning **Bottle 1** of **Wine 1 (W_1)** uniquely identifies a bottle.

"Generally, there are or have been **several bottles of the same wine** in the cellar. **For each wine, the bottles in the wine cellar of VINO are numbered.** For instance, the cellar has 20 bottles numbered 1 to 20 of a Semillon from 1996 named Rumbalara. **For documentation purposes, VINO may also want to record wines for which it does not own bottles.** The bottles are **either available in the cellar or they have been tasted and emptied**."

Bottles have **Number** and **Status** as its own attributes. But these are not enough to identify the bottles **UNIQUELY**.

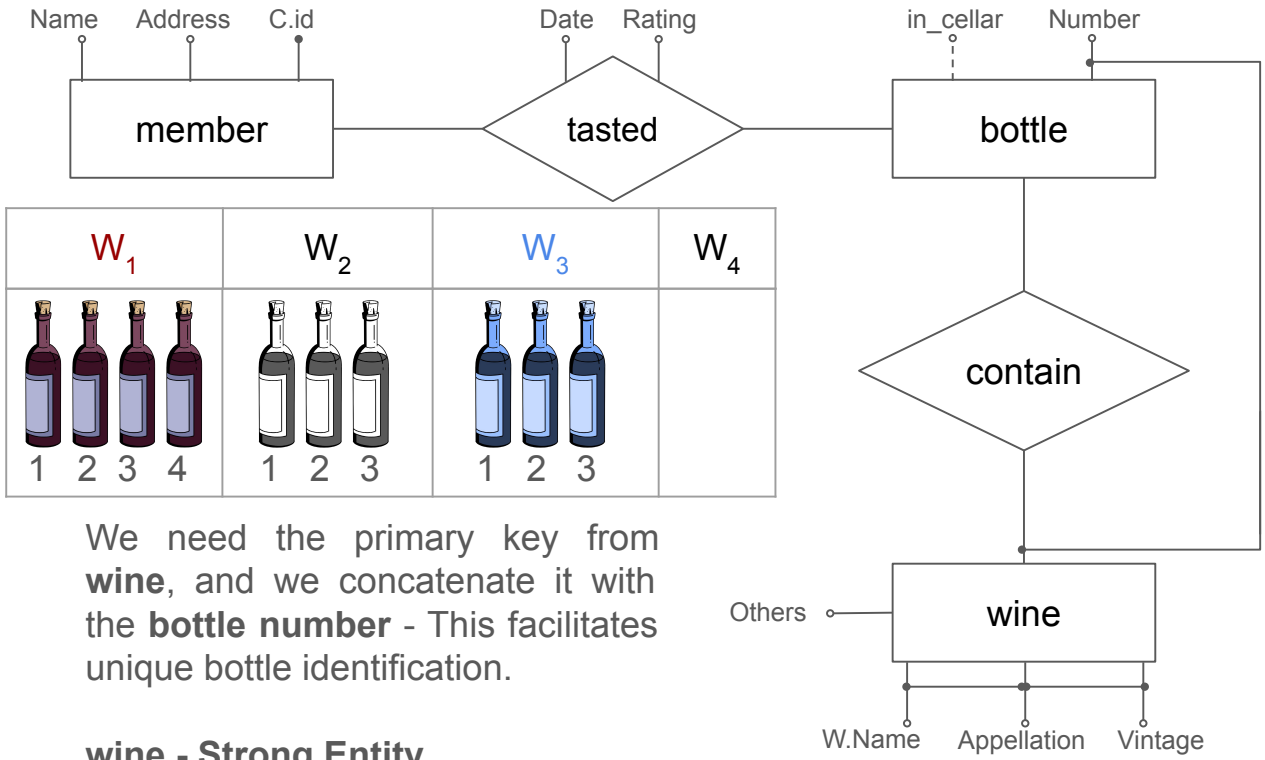


But, mentioning **Bottle 1** of **Wine 1 (W_1)** uniquely identifies a bottle.

Thus we need to identify the wine first, then the bottle - We need the primary key from **wine**, and we concatenate it with the **bottle number** - This facilitates unique bottle identification.

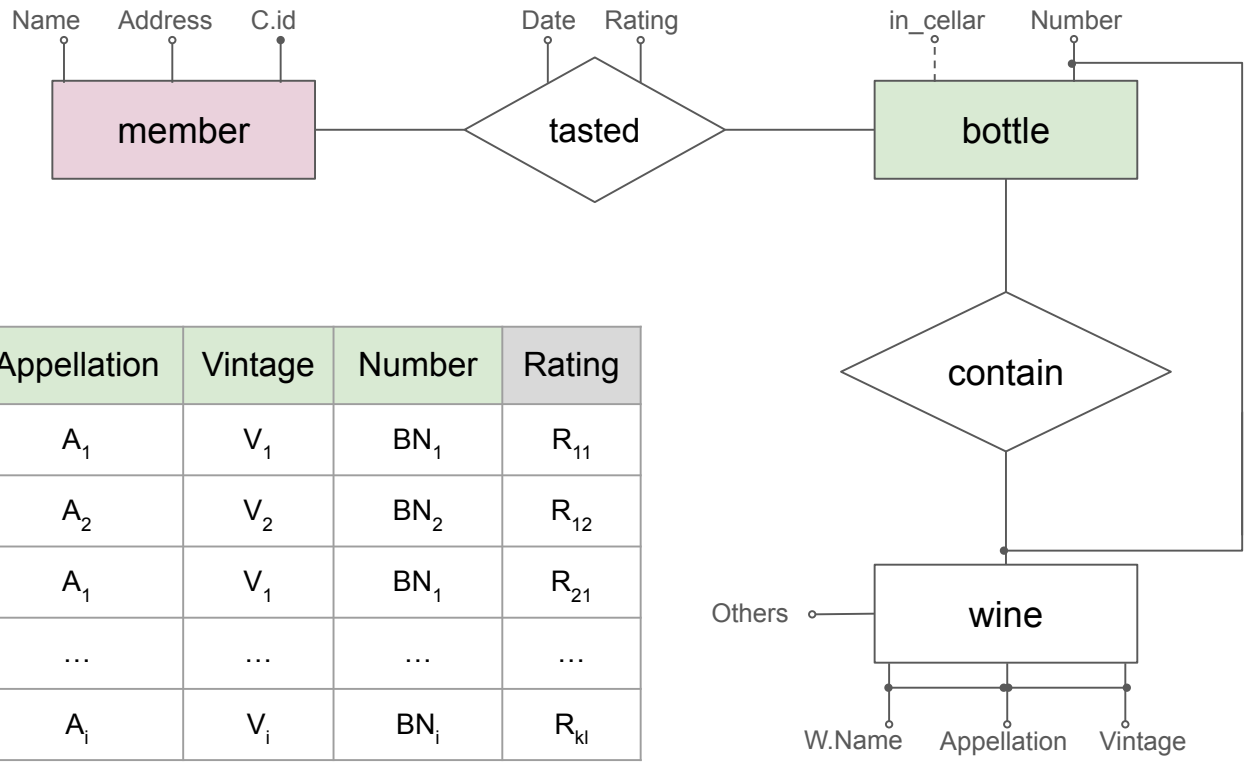
"Generally, there are or have been **several bottles of the same wine** in the cellar. **For each wine, the bottles in the wine cellar of VINO are numbered.** For instance, the cellar has 20 bottles numbered 1 to 20 of a Semillon from 1996 named Rumbalara. **For documentation purposes, VINO may also want to record wines for which it does not own bottles.** The bottles are **either available in the cellar or they have been tasted and emptied**."

Bottles have **Number** and **Status** as its own attributes. But these are not enough to identify the bottles **UNIQUELY**.



We need the primary key from **wine**, and we concatenate it with the **bottle number** - This facilitates unique bottle identification.

wine - Strong Entity
bottle - Weak Entity
Existence of a bottle of wine depends on the existence of the wine.

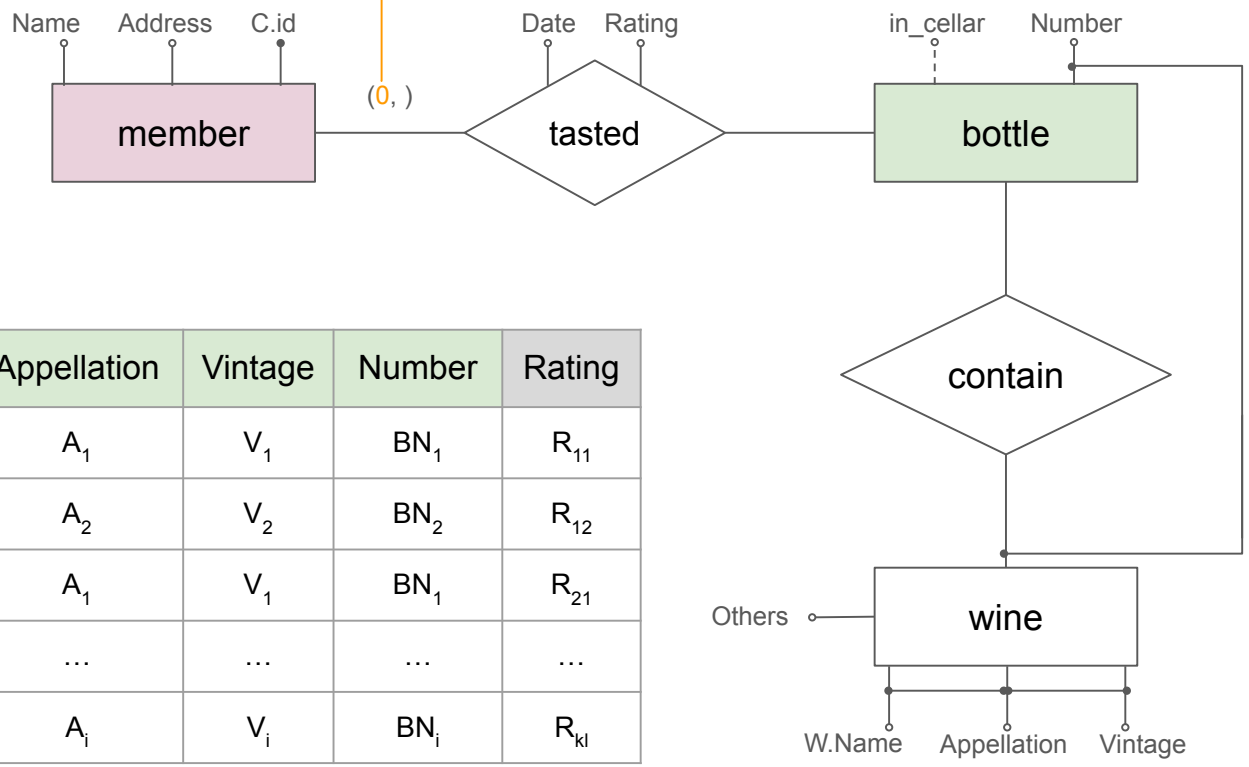


Date	C.id	Name	Appellation	Vintage	Number	Rating
D_1	C_1	N_1	A_1	V_1	BN_1	R_{11}
D_2	C_1	N_2	A_2	V_2	BN_2	R_{12}
D_3	C_2	N_1	A_1	V_1	BN_1	R_{21}
…	…	…	…	…	…	…
D_k	C_j	N_i	A_i	V_i	BN_i	R_{kl}

Example instantiation of the **tasted** relationship

There can be members who have **NOT** attended any session, thus have **NOT** tasted any bottle - Thus **DO NOT** appear in **Taste** relationship - Min constraint = 0

"At each session, **the attending members** taste several bottles". - **Suggests that there can be non-attending members.**

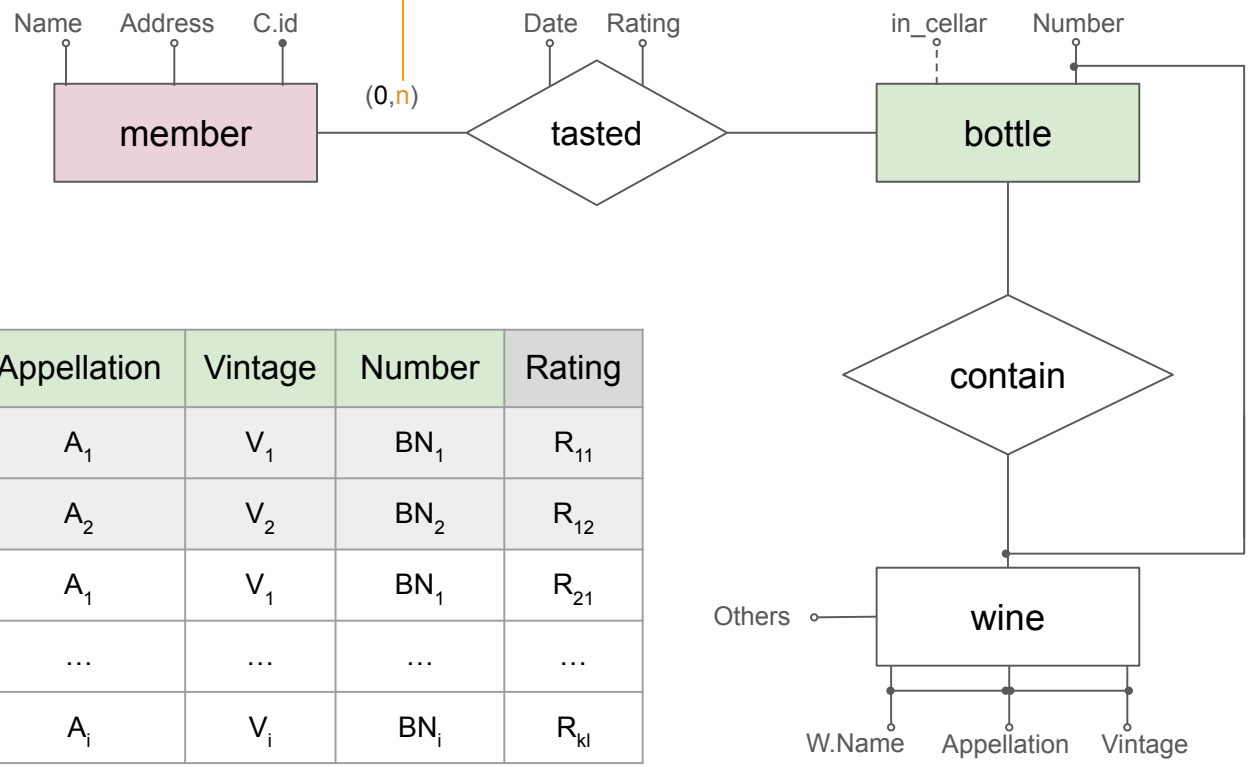


Date	C.id	Name	Appellation	Vintage	Number	Rating
D_1	C_1	N_1	A_1	V_1	BN_1	R_{11}
D_2	C_1	N_2	A_2	V_2	BN_2	R_{12}
D_3	C_2	N_1	A_1	V_1	BN_1	R_{21}
…	…	…	…	…	…	…
D_k	C_j	N_i	A_i	V_i	BN_i	R_{kl}

Example instantiation of the **tasted** relationship

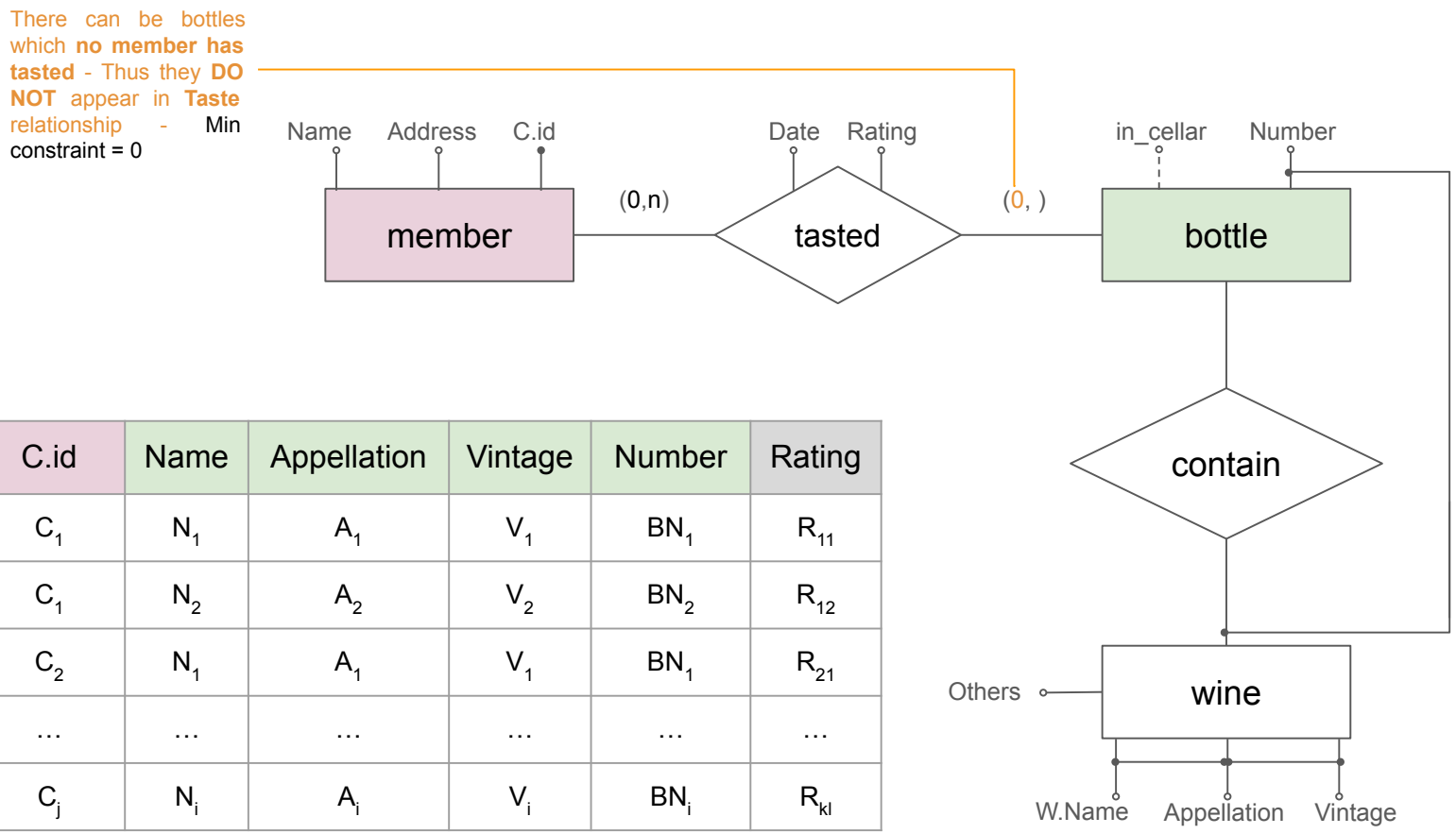
There can be members who have tasted more than one bottles - Thus can appear in **Taste** relationship **MORE THAN ONCE** - Max constraint = n (denotes **MANY**)

"At each session, the attending members **taste several bottles**." - **Suggests that there can be non-attending members.**



Date	C.id	Name	Appellation	Vintage	Number	Rating
D_1	C_1	N_1	A_1	V_1	BN_1	R_{11}
D_2	C_1	N_2	A_2	V_2	BN_2	R_{12}
D_3	C_2	N_1	A_1	V_1	BN_1	R_{21}
…	…	…	…	…	…	…
D_k	C_j	N_i	A_i	V_i	BN_i	R_{kl}

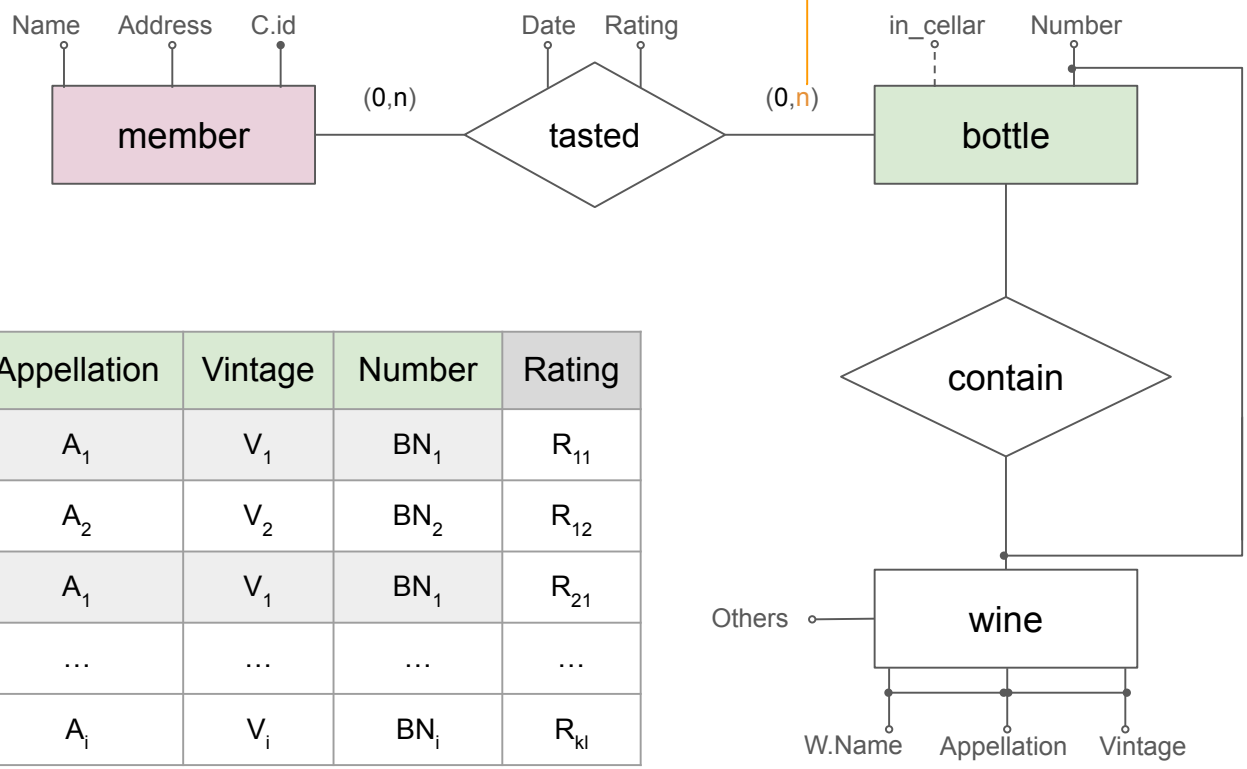
Example instantiation of the **tasted** relationship



Date	C.id	Name	Appellation	Vintage	Number	Rating
D_1	C_1	N_1	A_1	V_1	BN_1	R_{11}
D_2	C_1	N_2	A_2	V_2	BN_2	R_{12}
D_3	C_2	N_1	A_1	V_1	BN_1	R_{21}
…	…	…	…	…	…	…
D_k	C_j	N_i	A_i	V_i	BN_i	R_{kl}

Example instantiation of the **tasted** relationship

There can be bottles which **MANY members have tasted** - Thus they appear **MORE THAN ONCE** in **Taste** relationship - Max constraint = n (denotes **MANY**)

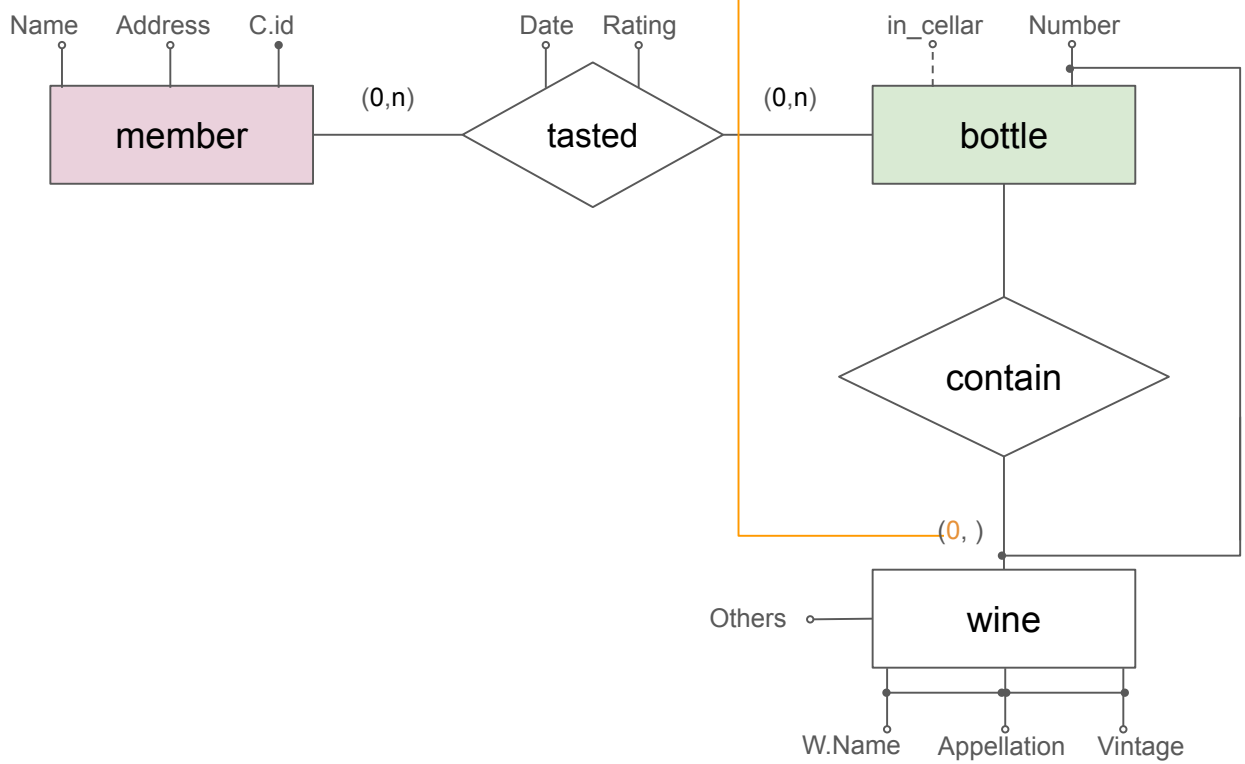


Date	C.id	Name	Appellation	Vintage	Number	Rating
D_1	C_1	N_1	A_1	V_1	BN_1	R_{11}
D_2	C_1	N_2	A_2	V_2	BN_2	R_{12}
D_3	C_2	N_1	A_1	V_1	BN_1	R_{21}
…	…	…	…	…	…	…
D_k	C_j	N_i	A_i	V_i	BN_i	R_{kl}

Example instantiation of the **tasted** relationship

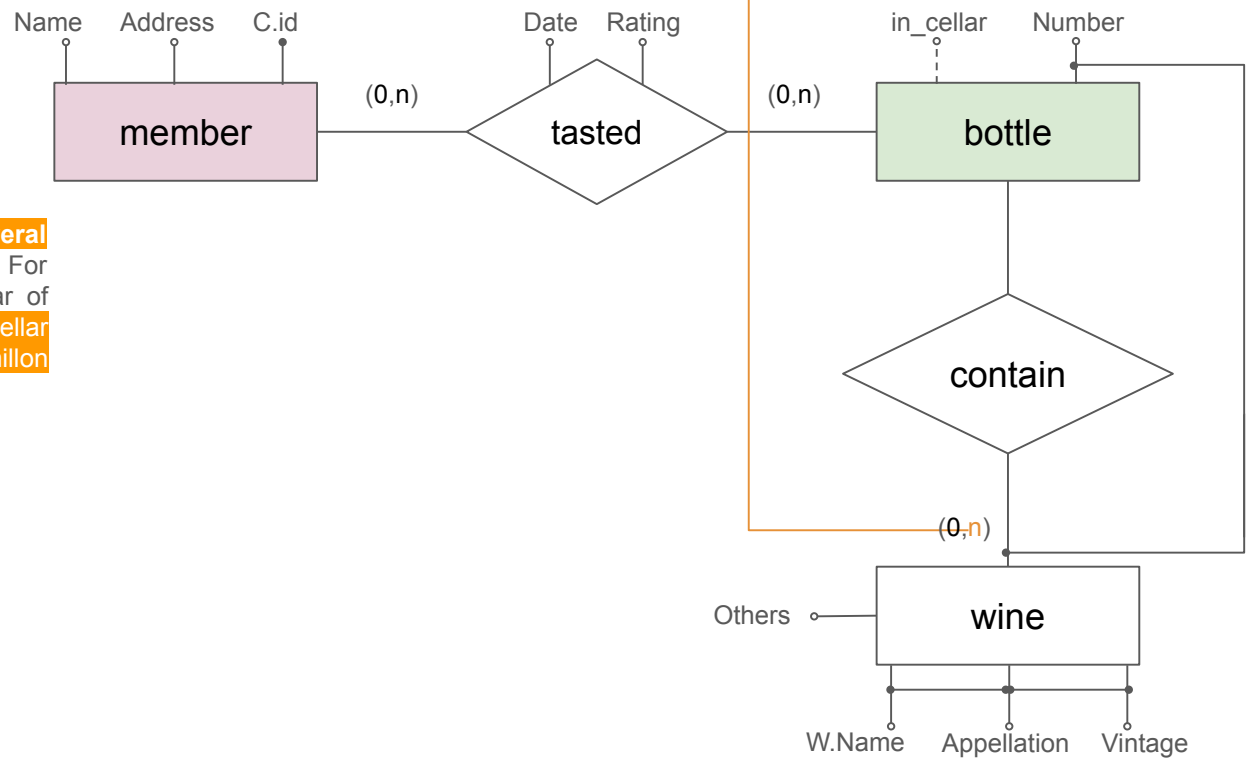
There can be wines, for which VINO has/had **no bottles** - Thus **DO NOT** appear in Contain relationship - Min constraint = 0

"For documentation purposes, VINO may also want to **record wines for which it does not own bottles.**"

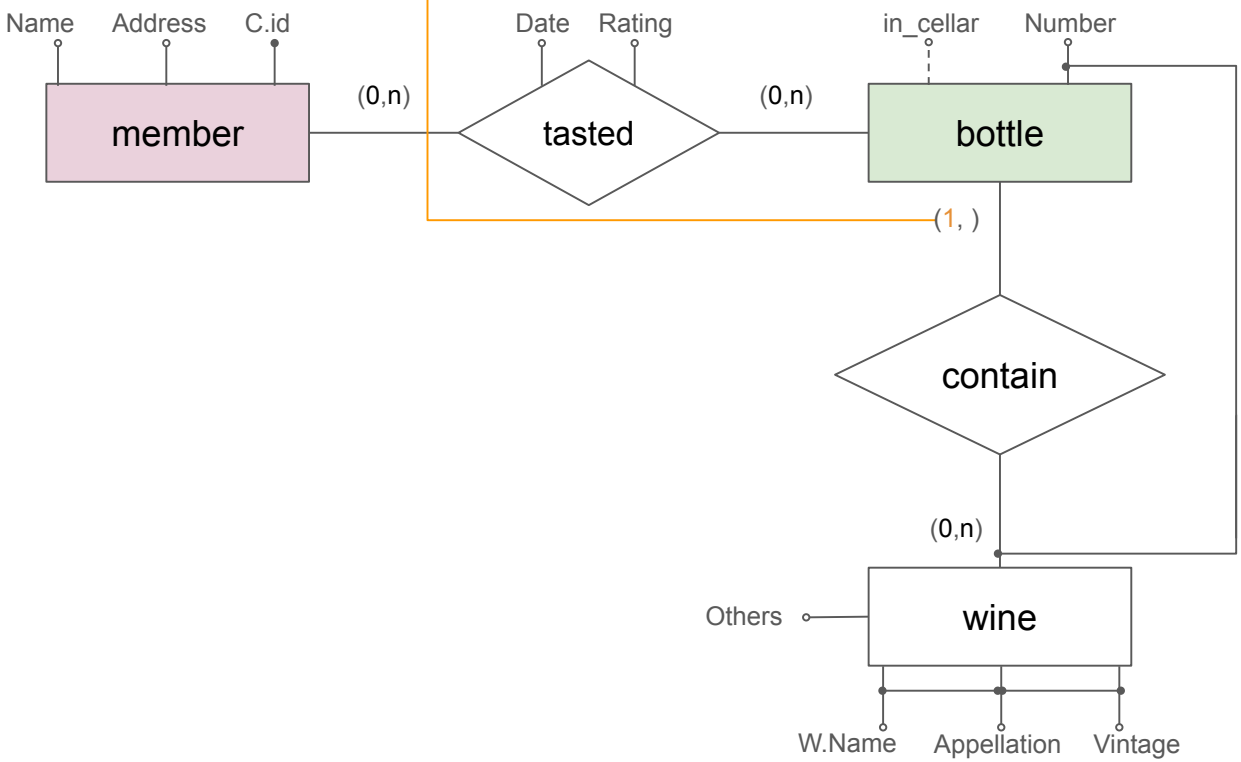


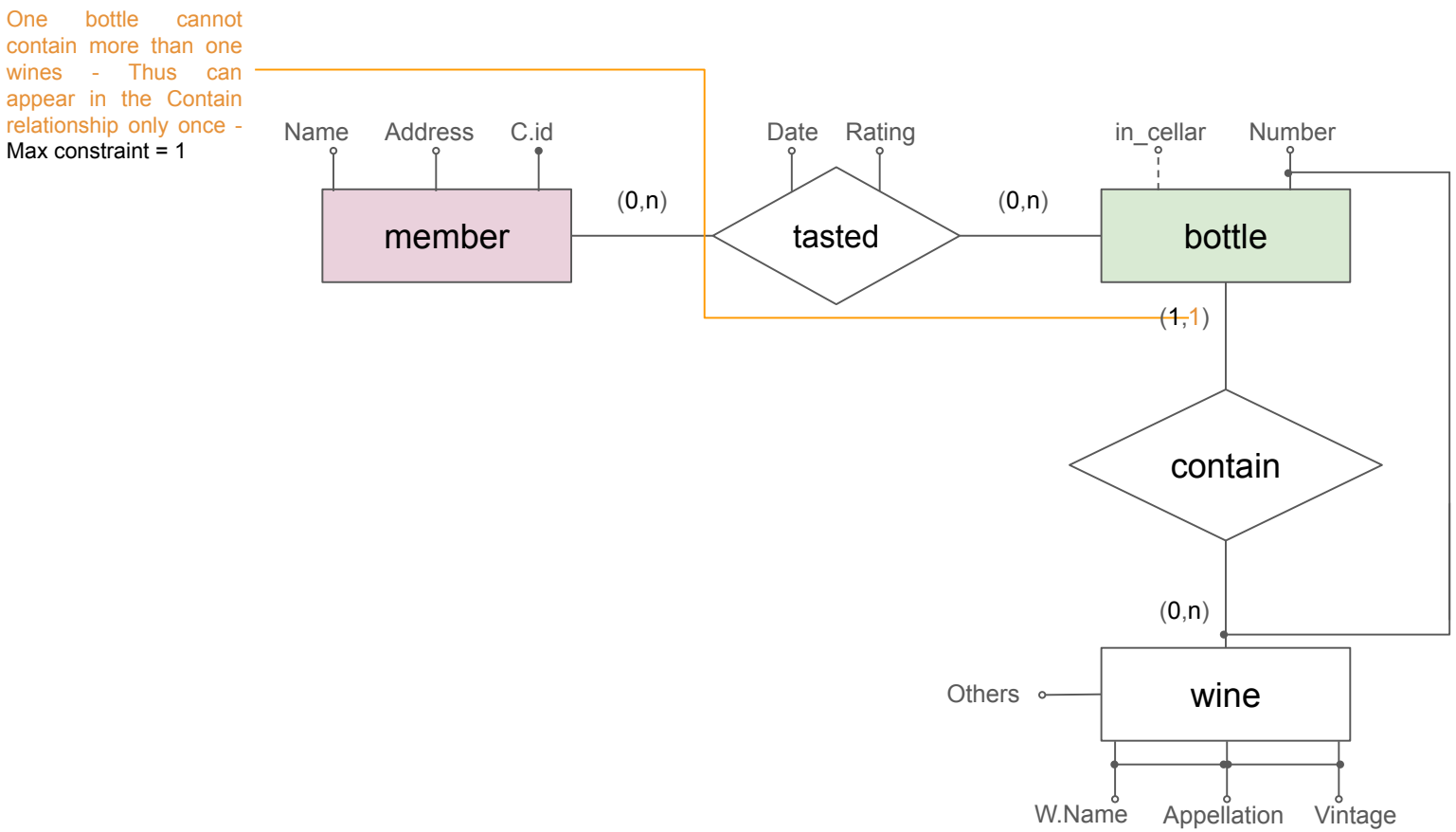
There can be **MANY** bottles of a wine - Thus can appear in Contain relationship **MORE THAN ONCE** - Max constraint = n(denotes **MANY**)

Generally, there **are or have been several bottles of the same wine in the cellar**. For each wine, the bottles in the wine cellar of VINO are numbered. For instance, the cellar has 20 bottles numbered 1 to 20 of a Semillon from 1996 named Rumbalara.

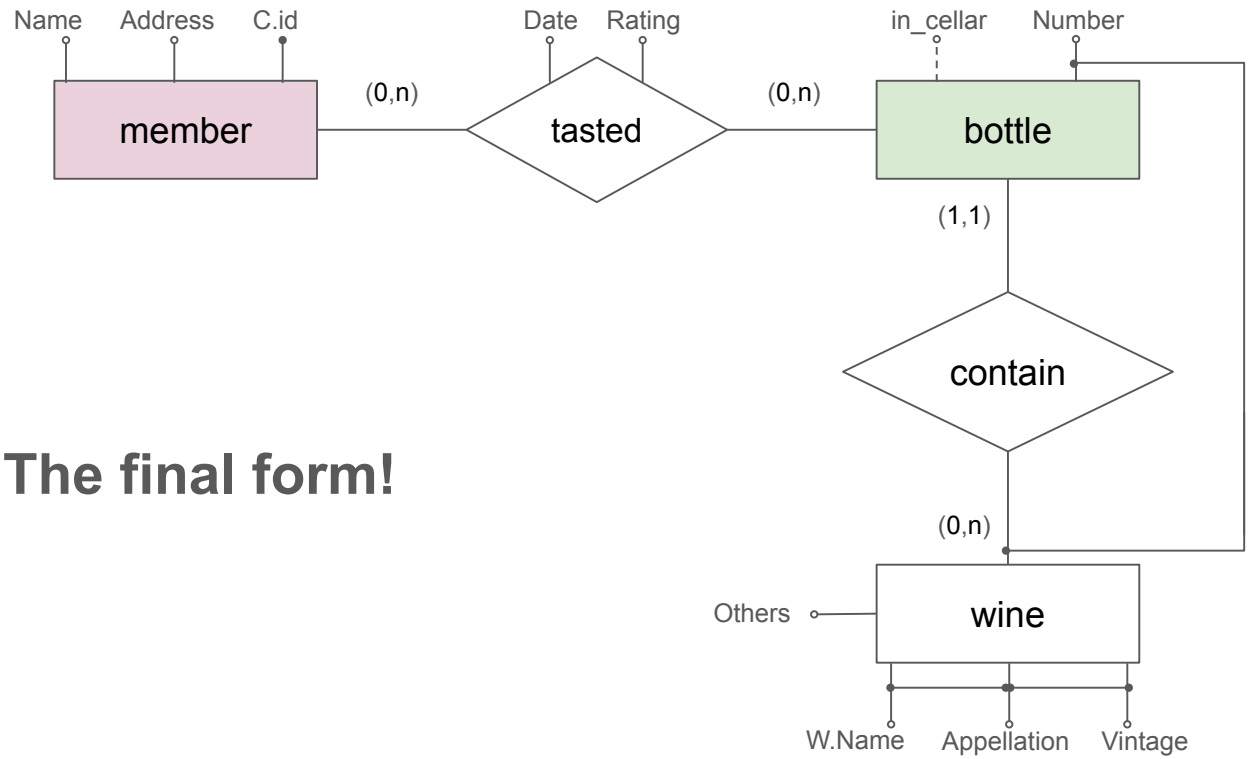


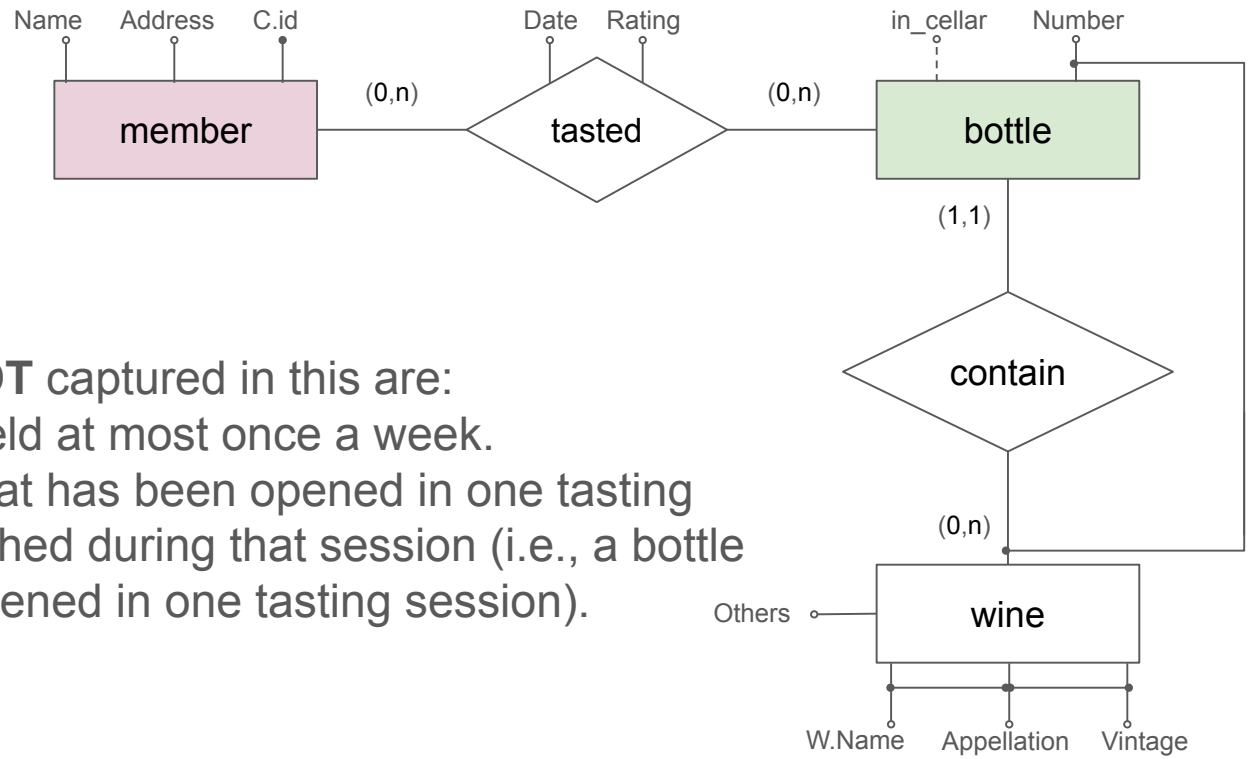
If there exists a bottle in the database, it has/had some wine - Thus it has to be in the Contain relationship - Min constraint = 1




One bottle cannot contain more than one wines - Thus can appear in the Contain relationship only once - Max constraint = 1
Name
Address
C.id
member
(0,n)
Date
Rating
tasted
(0,n)
in_cellar
Number
bottle
(1,1)
contain
(0,n)
Others
wine
W.Name
Appellation
Vintage

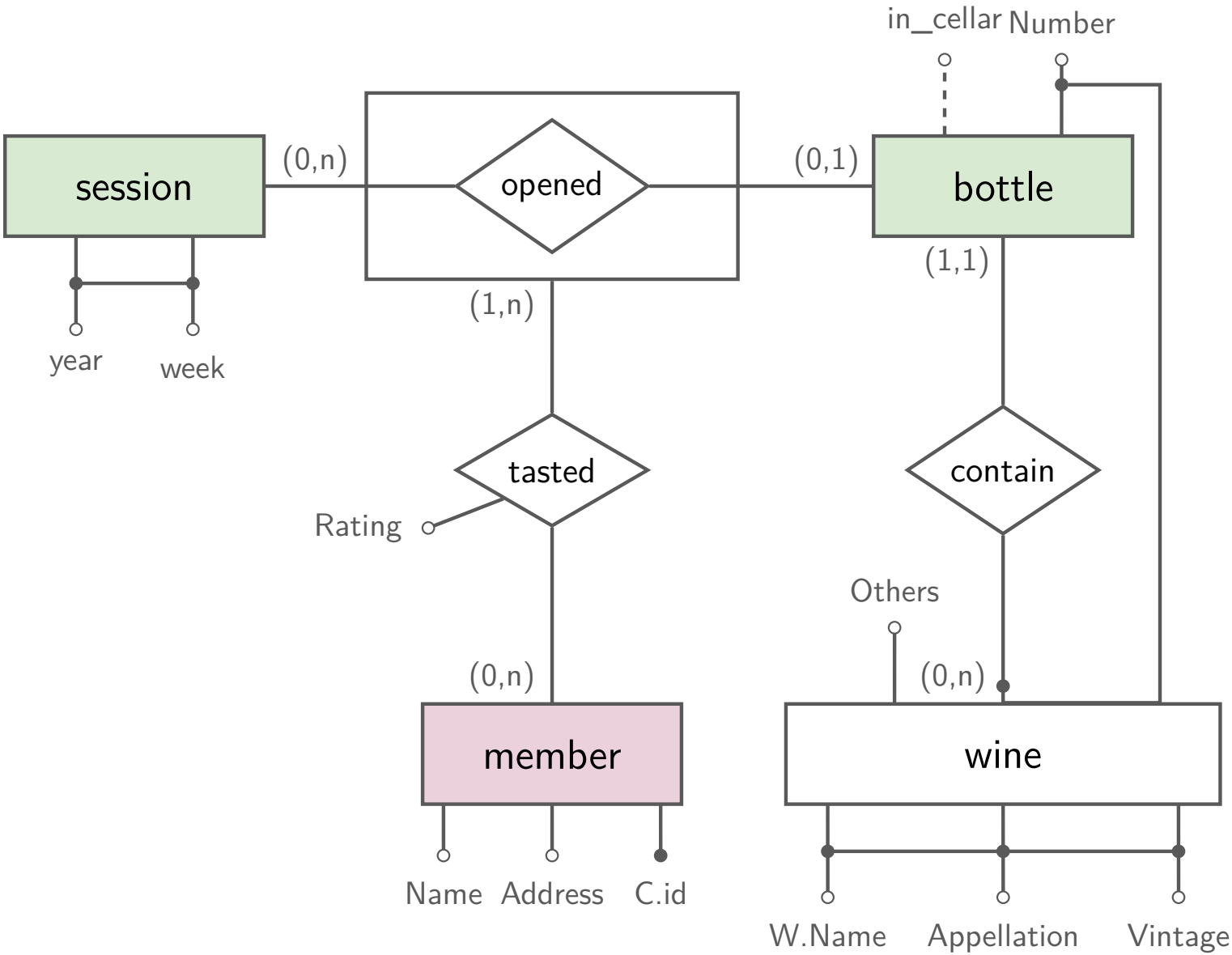
The final form!





The constraints **NOT** captured in this are:

- A session is held at most once a week.
- Every bottle that has been opened in one tasting session is finished during that session (i.e., a bottle can only be opened in one tasting session).


in_cellar
Number
session
(0,n)
opened
(0,1)
bottle
(1,1)
(1,n)
year
week
tasted
contain
Rating
Others
(0,n)
(0,n)
member
wine
Name
Address
C.id
W.Name
Appellation
Vintage

Q2. Logical Design

Translate the final ER diagram (with `session`, `open`, and aggregation) into a relational schema.

- Strong entities `member`, `wine`, `session`: attributes carry over, entity key becomes the primary key.
- Weak entity `bottle`: primary key = its partial key (`number`) concatenated with `wine`'s key.
- `contain` (bottle $\rightarrow$ wine, (1,1)): FK on `bottle`, absorbed into its own row.
- `open` (session $\leftrightarrow$ bottle, bottle side (0,1)): its own table, keyed by `bottle`.
- Aggregated `taste` (member $\leftrightarrow$ open): FK references `open`, not `bottle` directly.

Q2. Logical Design — member & wine

```sql
CREATE TABLE member (
  number  CHAR(10) PRIMARY KEY,
  name    VARCHAR(32) NOT NULL,
  address VARCHAR(64) NOT NULL
);

CREATE TABLE wine (
  name             VARCHAR(32),
  appellation      VARCHAR(32),
  vintage          INTEGER,
  alcohol_degree   NUMERIC NOT NULL,
  bottled          VARCHAR(128) NOT NULL,
  certification    VARCHAR(64),
  country          VARCHAR(32) NOT NULL,
  PRIMARY KEY (name, appellation, vintage)
);
```

Q2. Logical Design — session & bottle

```sql
CREATE TABLE session (
  year INTEGER,
  week INTEGER,
  PRIMARY KEY (year, week)
);
CREATE TABLE bottle (
  wine_name   VARCHAR(32),
  appellation VARCHAR(32),
  vintage     INTEGER,
  number      INTEGER CHECK (number > 0),
  PRIMARY KEY (number, wine_name, appellation, vintage),
  FOREIGN KEY (wine_name, appellation, vintage)
    REFERENCES wine (name, appellation, vintage)
);
```

(1,1) on bottle's contain → (`wine_name, appellation, vintage`) is NOT NULL by construction of the PK.

Q2. Logical Design — open

```sql
CREATE TABLE open (
  wine_name      VARCHAR(32),
  appellation    VARCHAR(32),
  vintage        INTEGER,
  bottle_number  INTEGER,
  session_year   INTEGER NOT NULL,
  session_week   INTEGER NOT NULL,
  PRIMARY KEY (bottle_number, wine_name, appellation, vintage),
  FOREIGN KEY (session_year, session_week)
    REFERENCES session (year, week),
  FOREIGN KEY (bottle_number, wine_name, appellation, vintage)
    REFERENCES bottle (number, wine_name, appellation, vintage)
);
```

Bottle (0,1) in open: a row exists only for bottles that have been opened → open's PK = bottle's key (at most one session per bottle).

Q2. Logical Design — taste (aggregate)

```sql
CREATE TABLE taste (
  wine_name     VARCHAR(32),
  appellation   VARCHAR(32),
  vintage       INTEGER,
  bottle_number INTEGER,
  member        CHAR(10),
  rating        VARCHAR(32) NOT NULL,
  PRIMARY KEY (member, bottle_number, wine_name, appellation, vintage),
  FOREIGN KEY (member) REFERENCES member (number),
  FOREIGN KEY (bottle_number, wine_name, appellation, vintage)
    REFERENCES open (bottle_number, wine_name, appellation, vintage),
  CHECK (rating IN ('very good', 'good', 'average',
                    'mediocre', 'bad', 'very bad'))
);
```

Referencing open (not bottle) is what the aggregation encodes: a rating is tied to a specific *opening event*, not just to a bottle.

Questions?
Drop a mail at: pratik.karmakar@u.nus.edu